



41. First Missing Positive

Brute Force

Understanding the `max_range + 1`

The key insight is: **the answer could be `n+1` itself.**

Consider this example with `n = 3`:

```
nums = [1, 2, 3]
```

- The array contains **all** of 1, 2, 3
- So the first missing positive is **4** (which is `n+1`)

If your loop only goes up to `n` (i.e., `range(1, n+1)` → checks 1, 2, 3), you'd never return 4 and the loop would just... end without an answer.

Why the answer is always $\leq n+1$

For an array of length `n`, it can hold **at most** `n` distinct positive integers. So among the integers `1, 2, 3, ..., n+1` (which are `n+1` values total), at least one must be missing.

This guarantees the answer is always in the range `[1, n+1]` — never higher.

What the loop is really doing

python

```
max_range = n + 1
```

```
for x in range(1, max_range + 1): # checks 1, 2, ..., n+1
```

```
if x not in num_set:
    return x
```

Scenario	Example (n=3)	Answer
Missing something early	[2, 3, 4]	Returns 1
Missing something middle	[1, 3, 4]	Returns 2
Contains exactly 1..n	[1, 2, 3]	Returns 4 (= n+1)

The loop **always returns inside the loop** — it's guaranteed to find a missing value before or at `n+1`, so you never actually need a return statement after it.

Your confusion summarized

| "we can stop at 3 as n = 3"

Only if you're sure the answer isn't 4. But `[1, 2, 3]` proves the answer *can* be `n+1`, so you must check up to and including `n+1`.

Changing Input Array

Let's trace through `nums = [3, 4, -1, 1]` step by step.

Step 1: Replace negatives with 0

```
[3, 4, -1, 1]
      ↑ negative, replace with 0
```

Result: `[3, 4, 0, 1]`

Step 2: Marking loop — `n = 4`, so valid indices are `0..3`

`i=0, nums[0]=3`

```
val = abs(3) = 3
idx = 3 - 1 = 2
```

```
nums[2] = 0 → sentinel case → nums[2] = -(4+1) = -5
```

```
[3, 4, -5, 1]  
      ↑ marked
```

i=1, nums[1]=4

```
val = abs(4) = 4  
idx = 4 - 1 = 3
```

```
nums[3] = 1 > 0 → nums[3] *= -1 = -1
```

```
[3, 4, -5, -1]  
      ↑ marked
```

i=2, nums[2]=-5

```
val = abs(-5) = 5  
idx = 5 - 1 = 4
```

```
idx >= n (4 >= 4) → SKIP ✓ (sentinel safely ignored)
```

i=3, nums[3]=-1

```
val = abs(-1) = 1  
idx = 1 - 1 = 0
```

```
nums[0] = 3 > 0 → nums[0] *= -1 = -3
```

```
[-3, 4, -5, -1]  
      ↑ marked
```

Step 3: Find first unvisited index

```
[-3, 4, -5, -1]
 0  1  2  3
```

x	idx	nums[idx]	>= 0?
1	0	-3	No → visited
2	1	4	Yes → return 2 ✓

Why index 1 (value 2) was never marked

2 was never in the original array, so nothing ever set `nums[1]` to negative — it stayed as 4. That's the whole idea: **unvisited = positive = missing number**.

The Risk

In the marking loop you do:

```
val = abs(nums[i]) # ← you take abs() of every element
idx = val - 1
```

So if you pick a sentinel like -1, watch what happens:

```
nums[idx] = -1 (sentinel)
```

Later, when i reaches that index:

```
val = abs(-1) = 1
```

```
idx = 1 - 1 = 0
```

→ you'd accidentally mark index 0 as visited! ❌

A sentinel like -5 has the same problem if 5 is a valid index (i.e., $5 \leq n$).

Why (n+1) is Safe

```
val = abs(-(n+1)) = n+1
```

```
idx = (n+1) - 1 = n
```

→ `idx >= n`, so it hits the bounds check and gets skipped ✓

Since `n` is out of bounds for a 0-indexed array of length `n`, it will **always be ignored** by the marking loop.

In Short

Sentinel	<code>abs(sentinel)</code>	Safe?
<code>-1</code>	<code>1</code> → valid index, causes wrong marking	✗
<code>-5</code>	<code>5</code> → might be valid index	✗ (depends on n)
<code>-(n+1)</code>	<code>n+1</code> → always out of bounds	✓

So `-(n+1)` isn't magic — it's just the smallest guaranteed-out-of-bounds negative value. Anything with `abs(sentinel) > n` would work equally well (like `-(n+2)`, `-(n+100)`, etc.).